

UE Software Engineering: Project Report

Data compressing for speed up transmission.

Arno LESAGE (la202985)



University: Université Côte d'Azur – EUR DS4H

Curriculum: Master 1 – Informatique, parcours Intelligence Artificielle

Lecture Instructor: Pr. Jean-Charles Régim

Hand-out date: 2nd November 2025

Abstract – In network protocols and telecommunications, transmission speed is a critical performance metric, especially for delay-sensitive applications. Reducing latency has long been a central challenge for the internet and remains an active area of research. Among the proposed solutions, compression techniques have emerged as effective ways to minimise transmission delays. This report explores the implementation of three such techniques in Python, followed by a comparative performance evaluation.

Table of Contents

Introduction	3
1 – Formalisation and definition of Bit packing	4
1.1 – Problem formalisation	4
1.2 – Requested versions of Bit Packing	4
2 – Implementation details	7
2.1 – Code structure and organisation	7
2.1.1 – Abstract Compressor class	7
2.1.2 – Bit Packing class and factory	8
2.1.3 – Organisation summarised	9
2.2 – “Split” implementation	11
2.3 – “Nosplit” implementation	13
2.4 – “Overflow” implementation	14
3 – Tests and Benchmarks	17
3.1 – Tests	17
3.2 – Benchmarks	18
3.2.1 – Plots	18
3.2.2 – Hypothesis Testing	22
Conclusion	26
Sources	27

Introduction

In the field of Network protocols and Telecommunication, where delay may affect some applications, transmission speed is considered a key metric. Optimising the former quickly became a topic in itself, one of the global challenges of the internet and remains to this day an actively researched field.

One of the unravelled problems was the optimisation of integer arrays transmission, which might become costly. Indeed, even today, most “normal” integers are stored in a 32-bit format. Unfortunately, this way of proceeding also implies the usage of thirty-two bits to store numbers that could have been stored in a smaller format, such as 16-bit, 8-bit, or even less.

This observation led to the creation of the “Bit Packing” concept, in other words, a new concept to speed up integer transmission. The principle remains simple though: instead of transmitting raw integer arrays as is, we transmit a compressed version, reducing the number of bits taken by each integer while still being able to access them in the right order.

In this report, we explore three different implementations of Bit packing and investigate their performance in reducing the transmission delay. In the first part (1), we formalise our problem and expose the requested Bit Packing versions to implement. In the second part (2), we cover each separate implementation. Finally, part three (3) compares them before concluding.

1 – Formalisation and definition of Bit packing

1.1 – Problem formalisation

Definition 1: Bit Packing – Let A_n^b be a b -bit signed integer array of containing n integers and $A[i]$ be a function $[i]: A_n^b \rightarrow \mathbb{Z}$ which returns the integer stored at position i in the array A_n . We define Bit Packing as a function:

$$\text{bitPacking}: A_n^b \rightarrow A_n^{b'}$$

such that the appearance order of the integers within the array remains the same after compression ($\forall i \in \llbracket 0, n-1 \rrbracket : A_n^b[i] = A_n^{b'}[i]$), the number of bits to encode the integers is less than or equal to the original array ($b' \leq b$), and the compression possesses a lossless decompression ($\text{bitPacking}^{-1}(\text{bitPacking}(A_n^b)) = A_n^b$).

Note that Bit Packing can be used outside the scope of signed integers to other data types or even other data structures, but we will emit the hypothesis along with the report that we only use signed integers and an array-like structure to store them.

Definition 2: Transmission Delay – Let $X \rightarrow Y$ be the representation of the physical linking from the router X to the router Y , let L be the bit-length of a packet and R be the throughput of the linking in bit.s^{-1} , we define the transmission delay d_{trans} as $d_{\text{trans}} = \frac{L}{R}$. Assuming no propagation delay through the link (instant sending-transmission), d_{trans} corresponds to the necessary time to send the whole packet from router X to router Y .

With regards to our definition of Bit Packing and transmission delay, our goal is to implement three different versions of Bit Packing to minimise the transmission delay.

1.2 – Requested versions of Bit Packing

In this subpart, we review the requested versions of Bit Packing we will implement in the next part of the report.

To begin with, let us consider a 32-bit signed integer $x \in \mathbb{Z}$, its natural representation lies in base ten, but it is stored in base 2.

Example 3: The following 32-bit integer array $A_4^{32} = \{-128, 0, 65982, 2478\}$ can be represented in base two as:

$$A_4^{32} = \left\{ \begin{array}{l} \text{---} \text{00000000000000000000000000000000} \text{0000 0000 1000 0000} \\ \text{00000000000000000000000000000000} \text{0000 0000 0000 0000} \\ \text{00000000000000000000000000000000} \text{0000 0001 1011 1110} \\ \text{00000000000000000000000000000000} \text{0000 1001 1010 1110} \end{array} \right\}$$

By observing this structure, we discover that it is not necessary to store up to thirty-two bits of information per integer. Indeed, we could have stored all the information on seventeen bits per integer, effectively reducing A_4^{32} to A_4^{17} , which corresponds, after calculation, to a total gain of sixty bits over the whole array.

Virtually, compressing this way means that we could save a whole 32-bit integer in storage and store the compressed array as:

$$A_4^{17} = \left\{ \begin{array}{c} \text{---0000 0000 0100 0000 0|000 0000 0000 0000} \\ \text{00|10 0000 0011 0111 110|0 0000 1001 1010} \\ \text{1110|0000 0000 0000 0000 0000 0000 0000} \end{array} \right\} = \left\{ \begin{array}{c} 4194304 \\ 540524698 \\ 3758096384 \end{array} \right\}$$

Where the grey bits are buffers, bits corresponding to the remaining number of bits necessary to form a 32-bit integer.

Remark 4: In this example, we used the notation A_4^{17} with $n = 4$. In the following parts of the report, n will always reflect the number of encoded integers and not the number of stored 32-bit integers.

Pseudo-algorithm 5: “Split” Bit Packing

The first version “**split**” corresponds essentially to the approach described above:

- Step 1:** Change base 10 representation to base 2,
- Step 2:** Find the number of bits required to store the biggest absolute integer,
- Step 3:** Concatenate all the bits of each number, truncated to the maximal number of bits required within a unique array,
- Step 4:** Pad the array with 0 to the next multiple of 32.

The created array contains all initial integers encoded with b' bits ($b' \leq 32$) and is represented using three 32-bit integers.

Remark 6: It is important to remember that when storing a signed number, a bit must be used to store the sign of the integer. This bit will be included in the computation of the maximal number of bits required to encode a number.

The second version, “**nosplit**”, is also the same as above, with the exception that if an b' -bit integer cannot be fully stored on the same 32-bit integer after concatenation, then it is stored in the next integer.

In *example three*, the “compressed” array would have been represented as:

$$A_4^{17} = \left\{ \begin{array}{c} \text{---0000 0000 0100 0000 0|000 0000 0000 0000} \\ \text{0000 0000 0000 0000 0|000 0000 0000 0000} \\ \text{1000 0000 1101 1111 0|000 0000 0000 0000} \\ \text{0000 0100 1101 0111 0|000 0000 0000 0000} \end{array} \right\}$$

In this case, no compression would have achieved. However, if the encoding size had been smaller, it could have been possible to stack multiple numbers per 32-bit integer.

Pseudo-algorithm 7: “Overflow” Bit Packing

The last version to be implemented is “**overflow**”. The principle remains fairly similar to “split”; however, this version implements a maximum bit threshold, for which, if exceeded, stores the exceeding value in an overflow area, otherwise the value is compressed using “split.”

Step 1: Change base 10 representation to base 2,

Step 2: Search for the biggest value $v_{\max}$ strictly under the threshold (in terms of bits),

Step 3: For each value in the array, use “split” from step 3 with the necessary number of bits $v_{\max}$ if the value is under the threshold; otherwise, stores it in the overflow area and add a reference in the compressed array at this position to remember the order,

Step 4: Pad the compressed array with 0 to the next multiple of 32.

In *example three*, the results with a threshold of twelve bits would have looked like this:

$$A_4^{10} = \{-0010\ 0000\ 00\ 00\ 0000\ 0000\ 00\ 00\ 0000\ 0000\}$$

$$A_{\text{overflow}}^{32} = \left\{ \begin{array}{l} 0000\ 0000\ 0000\ 0001\ 0000\ 0001\ 1011\ 1110 \\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1001\ 1010\ 1110 \end{array} \right\}$$

In this case, we have an array where integers are encoded on 9 bits plus 1 bit for the overflow reference, which translates into 32 bits for the compressed array and 64 bits for the overflow area; in other words, 96 bits or three 32-bit integers in total to represent this structure.

Note that nothing is stopping us from doing a bit packing, like “split” in the overflow area too; this is what is going to happen in the implementation.

Remark 8: For all the above representations, we did not consider the addition of a sign-bit; in consequence, the representations are simplifications of the implementation. Technical details will be discussed in the next part.

2 – Implementation details

In this part, we will discuss implementation details of each version of Bit packing, but first, let us have a look at the project structure and organisation.

2.1 – Code structure and organisation

To begin with, all the code was written in Python 1.12.0. This choice was made for several reasons, including personal affinity with this particular programming language, the presence of useful libraries (NumPy, SciPy, bitarray and PyTest) to implement our Bit packing versions and test them, as well as the presence of extensive documentation if needed throughout the project.

Considering all that, here is the code structure of this project:

```
C:.\n|  arrayGenerator.py\n|  benchmark.py\n|  main.py\n|  benchmarkAnalysis.r\n|  README.md\n|  tests.py\n|  └── Compressor\n|      |   AbstractCompressor.py\n|      |   NoSplitCompressor.py\n|      |   OverflowCompressor.py\n|      └── SplitCompressor.py\n|  └── img\n|  └── IN\n|  └── OTHERS\n|      requirements.txt\n|  └── OUT
```

In this tree, `arrayGenerator.py` is responsible for the creation of the input files, `benchmark.py` for the benchmark, `main.py` for the BitPacking class and `tests.py` for the tests. In `Compressor/`, you will find the implementation for all compressor methods (split, nosplit, overflow). Finally, you will find the input files within `IN/`, the output files (benchmark) in `OUT/` and the required libraries in `OTHERS`.

2.1.1 – Abstract Compressor class

As you may have noticed while reading the tree, all the compressors are based on an abstract class `Compressor` that can be found in `Compressor/AbstractCompressor.py`. This class declare three abstract methods that have to be implemented by its subclasses:

- {abstract} `compress(np.ndarray[int]) → tuple[Any]`, which takes an integer NumPy array as an argument and returns a tuple containing the compressed version of the array along with possible other information,
- {abstract} `decompress(tuple[Any]) → np.ndarray[int]`, which takes the tuple returned by `compress` as input and returns an uncompressed integer NumPy array,
- {abstract} `get(int, tuple[Any]) → int` which takes both a position within the uncompressed array and the tuple returned by `compress` as input. This method returns the integer at the requested position from the compressed array.

A last concrete protected method is implemented within this class:

- {concrete, protected} `_getMaxBitLength(np.ndarray[int]) → int`, which takes an integer array as input and returns as output the required bit length to encode the biggest absolute value in the array.

The other compressor classes then inherit this abstract compressor and must implement all the abstract methods to be valid. Additionally, by the signature of each function, we ensure the possibility for different subclasses to return different outputs (with different additional information on top of the compressed array).

Moreover, the initialisation of each class is not specified by this abstract class and can be free for all subclasses. This possibility also means that we will have to take this information into account to build our factory and our `BitPacking` class.

Remark 9: In Python, the size of integers is not limited to 32 bits, but is rather unfixed; this is why we will use the `np.int32` type from NumPy instead of the built-in Python type `int` throughout the project. For simplicity, we will still use the notation “**int**” in this report, but this will correspond to “**np.int32**” in the real implementation.

2.1.2 – Bit Packing class and factory

The `BitPacking` class, contained within `main.py`, is the main class in direct contact with the user. This class aims to provide control over the compressor and a simple interface for the user. This class provide three private attributes:

- {private} `__compressor:Compressor`, which takes whatever `Compressor` inheriting from the Abstract Compressor,
- {private} `__arr:np.ndarray[int]`, which stores the current uncompressed integer NumPy array (either the one after decompression or the one before compression, depending on whether decompression happened),
- {private} `__compressed:tuple[Any]`, which stores the compressed array and the information relative to it.

The initialisation of the `BitPacking` class is done by feeding the constructor with a mode which can be either “**split**”, “**nosplit**” or “**overflow**” and represents the compressor to use. If necessary, the constructor also lets the user add parameters like the threshold for the “**overflow**” constructor.

The constructor then sends the information to the function `main.createCompressor`, which acts as a factory before declaring the other private attributes.

The `BitPacking` class also provides access to the private attributes through two setters and two getters:

- {getter} `getArr()` → `np.ndarray[int]`, which returns the value of the private attribute `__arr`,
- {getter} `getCompressedArr` → `tuple[Any]`, which returns the value of the private attribute `__compressed`,
- {setter} `setArr(np.ndarray[int])`, which sets the value of the private attribute `__arr`,
- {setter} `changeMode(str)`, which changes the compressor in use and the value of the corresponding private attribute `__compressor`.

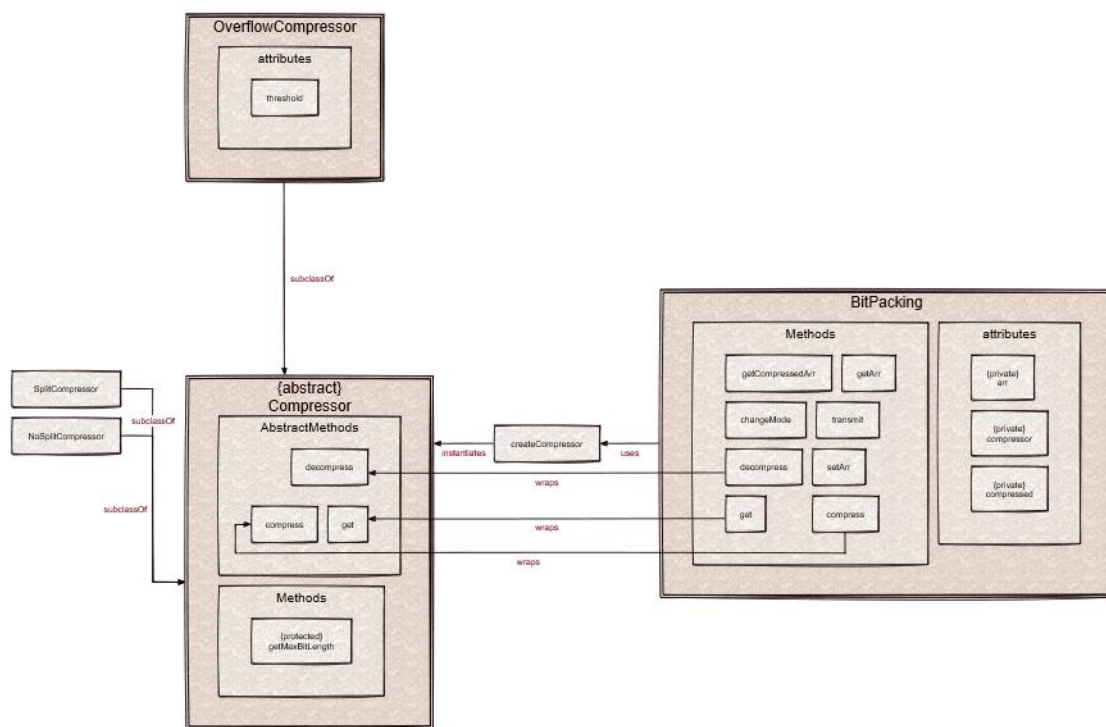
Nevertheless, the important part of the `BitPacking` class remains its wrappers around the methods present within the `AbstractCompressor.Compressor` class. This way, we can directly execute the following methods from the `BitPacking` class without worrying about which compressor we are using:

- {wrapper} `compress()` → `None`, which compresses the integer array we fed the `BitPacking` class through the setter `setArr` before storing the compressed array internally,
- {wrapper} `decompress()` → `None`, which decompresses the compressed array stored internally before overwriting the value of the private attribute `__arr`,
- {wrapper} `get(int, bool)` → `int`, which, given a position and a boolean stating if we get the value from the compressed array or the uncompressed array, returns the value at the given position from either the compressed or uncompressed array.

Finally, a last method `transmit(bool)` → `None` is provided to mimic the transmission of either the compressed or uncompressed array locally. This method serialises the data, writes them in a buffer and reads the buffer to mimic reception. This method is not normally used, but can be used to measure performance on a benchmark.

2.1.3 – Organisation summarised

We can now summarise the linking of every class using this schema:



Note that, even though external libraries were used in the project, we did not mention them in the above schema. However, they are still involved; this is why we will take a bit of this part to explain why we are using these libraries.

In this project we used ten libraries including four externals (NumPy, SciPy, PyTest and bitarray) as well as six built-in modules (time, os, pickle, typing, io and abc). Here are the reasons which pushed us to use them:

- **NumPy:** Used for the inclusion of the `np.int32` type to work with 32-bit integers, inclusion of a random number generator to generate and assess large arrays, and the availability of an array type which greatly differs from the built-in list implementation.
- **SciPy:** Used to generate random numbers following specific distributions (Boltzmann distribution, also known as truncated Planck distribution or discrete exponential distribution) to assess bit packing with different contexts.
- **PyTest:** Widely recognised as the “default” testing solution in Python, it was used to check whether the different bit packing algorithms were well-implemented.
- **bitarray:** Used to optimise the memory usage of the compressed integer array (which is essentially an array of bits) compared to the standard solution in Python or NumPy.
- **time:** Used to benchmark our process.
- **os:** Used to loop through IN/ folder and perform the tests automatically.
- **pickle and io:** Used to mimic the transmission of data through a network using serialisation and buffering.

- `abc`: Used to implement abstract classes in Python.
- `typing`: Used for documentation and function signatures.

2.2 – “Split” implementation

In this subpart, we will present the implementation for the “**split**” version of Bit packing. As explained earlier, this compressor inherits from the abstract compressor and therefore must implement the methods `compress`, `decompress` and `get`. In the following, we will describe the implementation of each method.

The first method we will describe is `compress`. This method takes a NumPy array of integers as input and returns a tuple containing, in the first position, the compressed array, in the second position, the required bit length to encode the absolute biggest value of the array and in the last, the length of the uncompressed array.

Pseudo-code 10: Split compression

```

1 def compress(self, arr:np.ndarray[int]) → tuple[bitarray.bitarray, int, int]:
2   maxBitLength ← self._getMaxBitLength(arr)
3   compressedArr ← 0-full bitarray of size  $32 \left\lceil \frac{n \cdot (\text{maxBitLength} + 1)}{32} \right\rceil$ 
4   Foreach i, element in enumerate(arr) do:
5     Transform element into binary and truncate to maxBitLength
6     If element < 0 then compressedArr[i(maxBitLength + 1)] ← True else False
7     Foreach ibit, bitVal in enumerate(element) do:
8       compressedArr[i(maxBitLength + 1) + ibit + 1] ← bitVal
9   return compressedArr, maxBitLength, length(arr)

```

If we refer to the *pseudo-algorithm five*, **step 2** (finding the maximum required bit-length) is done at line two, then we initialise our compressed array with zeros and the particular size:

$$32 \left\lceil \frac{n \cdot (\text{maxBitLength} + 1)}{32} \right\rceil$$

Considering an n -sized array of b -bits encoded integer A_n^b , in this formula, $(\text{maxBitLength} + 1)$ corresponds to the number of bits required to represent the biggest number, plus a bit to determine the sign of the compressed number. This value times the size of the original array n can be interpreted as the number of bits required to encode the whole compressed array. Divided by the number of bits to encode the uncompressed array $b = 32$ and ceiled, we obtain the required number of 32-bit integers to store the data. Multiply by $b = 32$ again and you will find the number of bits required to represent the compressed array (padding to the next multiplier of b included, **step 4**).

After that, we transform every base ten number into a truncated binary number (**step 1, 3**) before inserting them in the compressed array at the right position while ensuring the encoding of the sign before inserting it.

The method ultimately returns the tuple mentioned.

Remark 11: For all the compressors, a global constant can be set to move from 32-bit to other formats. By consequence, it is possible in theory to encode on 64-bit integers, for example.

Now, we can move on to the decompression method `decompress`. This method takes, according to its signature, a variable number of entries (`*args`), but in reality, it is just, in this case, a gimmick notation. Indeed, the real entry consists only of the tuple generated by the `compress` method. In the end, the method returns a NumPy array of integers.

Remark 12: Due to the `*args` notation in the signature, we are obliged to unpack the entry when passing the values to the function. For example, if `val` is the tuple generated by `compress`, then to decompress the array, we use `decompress(*val)` instead of `decompress(val)`. This may be misleading, but this notation avoids us from separating the information relative to the compressed array into different variables, which may become heavy syntactically.

Pseudo-code 13: Split decompression

```
1 def decompress(self, *args) → np.ndarray[int]:
2     Retrieve cArr, maxBitLength and initialLength from args
2     arr ← [] #Python list
3     i ← 0
4     While True do:
5         window ← Slice from  $i(\text{maxBitLength} + 1)$  to  $(i + 1)(\text{maxBitLength} + 1)$ 
6         If window out of cArr bound or  $\text{length}(\text{arr}) \geq \text{initialLength}$  then break
7         val ← cArr[window] #end excluded
8         elem ← '-' if val[0] else '' #sign bit
9         Concatenate elem with val[1:] #all element of val after index 1
10        arr.append(int(elem, base=2)); i++
11    return np.array(arr)
```

In this pseudo-code, we manage our decompression using a while loop and a window. The principle is quite simple, while the array is not fully decompressed (we do not have the same number of elements as in the original array), we create a window encompassing a number encoded over $\text{maxBitLength} + 1$ (b' plus the sign bit), decode the value in the window, update the uncompressed array, move the window by $b' + 1$ and repeat until we finish decompression.

Remark 14: The decompression, compression and get methods coded for each of the compressors are quite similar to each other; however, you will find some differences in the implementation due to the development time, which spans over multiple days (for example, `overflow` uses a for loop for decompression instead of a while loop.)

Finally, the last method `get` takes an integer `i` indicating the position we want to look at, as well as the tuple we mentioned for the decompression. For the implementation of this method, the main “challenge” was to translate the position of the user to the position in

the compressed array. In the case of a split, this is quite simple; the position in the compressed corresponds to the value $i(\text{maxBitLength} + 1)$.

When we have this value, we then just need to take a slice of size `maxBitLength` from this position, decode the value obtained (by considering the first bit as a sign bit), and finally translate it into base ten.

2.3 – “Nosplit” implementation

In this subpart, we will present the implementation for the “**nosplit**” version of Bit packing. As in the previous part, we will explain the main function of the compressor.

The first function `compress` takes the same argument as earlier, but results in a different tuple output. In the case of “**nosplit**”, the bit sign is managed outside of the main compressed array in a separate bit array. The output is therefore the same as earlier, plus the sign bitarray.

Remark 15: The usage of an additional array to store the sign bits was motivated by the split-avoiding nature of this structure. Indeed, if the sign was managed within the compressed array, we would have been limited to an integer of 31 bits, because a 32-bit integer would have been stored (in the compressed array) on 33 bits plus (32 bits + 1 bit for the sign), resulting in a cut.

Here is how the compression function works:

Pseudo-code 16: Nosplit compression

Let the following variable be:

- $\text{capacity} \leftarrow \left\lfloor \frac{32}{\text{maxBitLength}} \right\rfloor$: the value representing how many b' -bit encoded integer can contain a 32-bit integer.
- $\text{criticalIndex} \leftarrow \text{maxBitLength} \cdot \text{capacity}$: the value representing the index in a 32-bit integer at which we have to move to another integer.
- $\text{criticalShift} \leftarrow 32 - \text{capacity} \cdot \text{maxBitLength}$: the value representing the shift to apply in the compressed array to move to the next integer (this shift is constant through the same compression).
- n be the length of the original array.

```

1 def compress(self, arr:np.ndarray[int]) → tuple[bitarray, bitarray, int, int]:
2   maxBitLength ← max(self._getMaxBitLength(arr), 1)
3   Initialise capacity, criticalIndex, criticalShift and n
4   compressedArr ← 0-full bitarray of size  $32 \left\lceil \frac{n}{\text{capacity}} \right\rceil$ 
5   signArr ← 0-full bitarray of size  $n$ 
6   intNo ← 0
7   Foreach i, element in enumerate(arr) do:
8     Transform element into binary and truncate to maxBitLength
9     pos ←  $i \cdot \text{maxBitLength} + \text{intNo} \cdot \text{criticalShift}$ 

```

```

10     If pos in critical area then intNo++, pos ← pos + criticalShift
11     If element < 0 then signArr[i] ← True else False
12     Foreach ibit, bitVal in enumerate(element) do:
13         compressedArr[pos+ibit] ← bitVal
14     return compressedArr, signArr, maxBitLength, length(arr)

```

In this compression, the main goal is to adapt our index to the critical area (where no integer should be present to avoid splitting). For this, we compute a critical shift representing a value to add to a global shift every time we find ourselves in the critical area; you can find the implementation on lines 9 and 10.

Apart from this, nothing really changes with respect to the split compression.

The next method we will present is the `get` function. It takes the same arguments as the `splits get` (with the `nosplit` tuple instead of the `split` tuple, though), but finding the right position is much harder.

In this situation, we have two cases depending on the capacity:

- If the capacity is equal to one, then we know each integer is stored at indices multiple of thirty-two. In consequence, finding the integer we look for is easy and just computed as $32i$.
- If the capacity is greater than one, then multiple integers can be stored on a single 32-bit integer. As a consequence, we need to take this change into account if we look for an integer in the middle of a 32-bit integer. In this situation, we compute the position to look at as:

$$\text{pos} = \text{maxBitLength} \cdot (i \bmod \text{capacity}) + 32 \left\lfloor \frac{i}{\text{capacity}} \right\rfloor$$

Where the first parts account for the number of bits to move within the 32-bit integer, and the second part accounts for the number of integers to move on.

When we have the starting position, we can proceed exactly as in **“split”** `get`, but by considering that the sign is in the `signArr`.

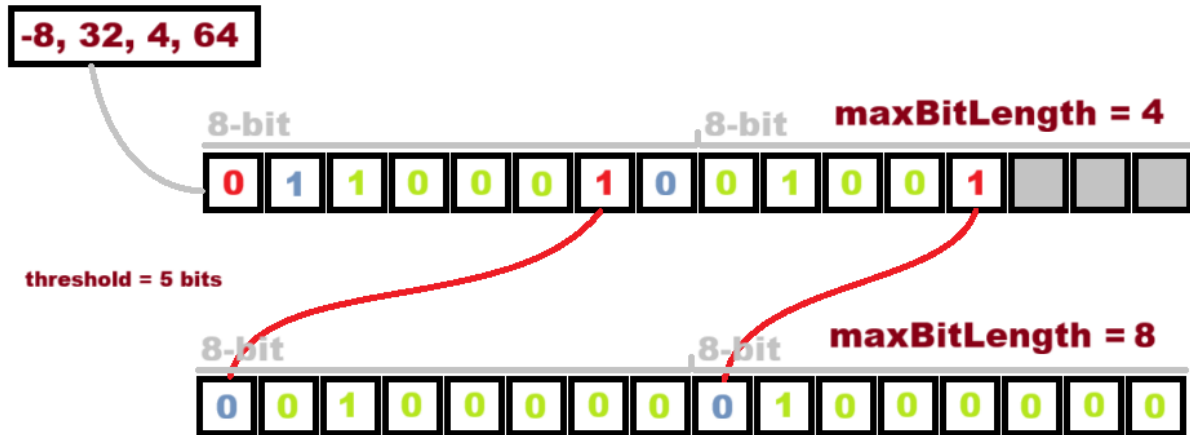
The last method to present is then `decompress`. It also takes the same argument as **“split”** `decompress`, but with the **“nosplit”** tuple. Here, the method is by far the simplest: use `get` iteratively with $i \in [0, n[$ and returns the resulting array.

2.4 – “Overflow” implementation

Finally, we can present the implementation of **“overflow”** BitPacking. For this implementation, we will let the user choose, when initialising the compressor, a particular value for which an integer will be passed on the overflow area instead of being compressed in the main array.

Remark 17: Even though it was not really asked for in the specification, we primarily understood that the overflow area must have been compressed as well. As a

consequence, and for time reasons, the overflow area is still compressed in the definitive version.



In this format, we have the first bit indicating whether the value is in the overflow area or not, the second bit, representing the sign, is read either in the overflow area or in the main area, depending on the previous bit. Finally, the integer is written in the corresponding area.

In the example above, where integers are encoded on 8 bits, all integers greater than or equal to $2^5 = 32$ is thrown in the overflow area and compressed with the maximum number of bits required to represent the biggest absolute value in the overflow area and vice versa.

Considering all of this here is how the `compress` method works:

Pseudo-code 18: Overflow compression

```

1 def compress(self, arr: np.ndarray[int]) → tuple[bitarray, int, bitarray, int, int]:
2     maxBitLength ← self._getMaxBitLength([arr[arr < 2threshold]])
3     maxOBL ← self._getMaxBitLength([arr[arr ≥ 2threshold]])
3     compressedArr ← 0-full bitarray of size  $32 \left\lceil \frac{n \cdot (\max\text{BitLength} + 2)}{32} \right\rceil$ 
4     OArr ← 0-full bitarray of size  $32 \left\lceil \frac{n \cdot (\max\text{OBitLength} + 1)}{32} \right\rceil$ 
5     OCount ← 0
6     Foreach i, element in enumerate(arr) do:
7         Transform element into binary and truncate to maxBitLeng
8         pos ←  $i(\max\text{BitLength} + 2) - \text{OCount} \cdot (\max\text{BitLength} + 1)$ 
9         If element ≥ 2threshold then
10             compressedArr[starPos] ← True
11             Perform splitCompression with pos = OCount · (maxOBL + 1), OArr, maxOBL
12             OCount++; continue;
13         compressedArr[starPos] ← False
14         Perform splitCompression with pos = startPos + 1, compressedArr, maxBitLength
15     return compressedArr, maxBitLength, OArr, OMBL, length(arr)

```

As we can see, the function takes the same arguments as in other compressors (plus the threshold defined when initialising the object) but returns an overflow area and the max bit length of the biggest absolute value in the overflow area.

The main challenge in this implementation was to find the offset generated by the format. This offset was found to be:

$$\text{pos} = i(\text{maxBitLength} + 2) - \text{OCount} \cdot (\text{maxBitLength} + 1)$$

Where the first half controls how far we should have placed the value in the compressed array if no overflow was present, and the second half applies a correction depending on the number of overflows that occurred.

Then, the process is essentially the same as in “**split**” compression, but either performed in the compressed area or in the overflow area.

The `decompress` method is essentially the same as the compression method, but in reading instead of writing mode, the main goal is just to parse where the overflows are placed. The reading is then performed by a similar algorithm that is present in the “**split**” compressor.

Finally, the `get` method follows the same principles as the other `get`:

Step 1. Find the position to look at by iterating $\text{arrIndex} \in [0, n[$, computing the current position $p = \text{arrIndex}(\text{maxBitLength} + 2) - \text{OCount} \cdot (\text{maxBitLength} + 1)$ and increasing the overflow counter if the current position indicates an overflowed integer until reaching $\text{arrIndex} = i$. This approach ensures that the integer correctly aligns with the slicer we will use in step 2.

Step 2. Apply “**split**” `get` into either the overflow area or the compressed area at the found p .

Step 3. Decode the value and return it as in other `get` versions.

Now that we have seen all the implementation parts, we can see the tests and benchmarks.

3 – Tests and Benchmarks

In this part, we will evaluate and benchmark our compressor using PyTest and a custom Benchmark environment that will record the time spent for each function used.

3.1 – Tests

For more details or information about how to run the tests or how they are implemented, refer to the README.md or the file tests.py. In this section, we will explain what tests we made to ensure the correct execution of our compression process.

The **first test** we thought about was the following: “If I compress my data, will I be able to retrieve them back using decompression without loss or shifts?”.

This test is easy to set up: just take an array, make a copy of it, compress the copy, decompress the copy, and check if the copy is still the same as the original array.

The second test checks the following: “If I compress my data, will I be able to decode the element at a particular position without decompressing the whole array?”

For that, we have three cases:

Case 1. The position is in the middle of the array,

Case 2. The position is in the first position,

Case 3. The position is at the last position.

To evaluate this, we take an array, make a copy of it, compress the copy, use get on the positions depicted by the three cases above and compare them to the values at the same position in the original array.

Then, these two tests are run using fifteen randomly generated integer datasets (see arrayGenerator.py), plus three other special cases we identified:

- Small int (between $-2^8 + 1$ and $2^8 - 1$), with $n \in \{10^3, 10^5, 10^6, 4 \cdot 10^6\}$,
- Large int (between $-2^{16} + 1$ and $2^{16} - 1$), with $n \in \{10^3, 10^5, 10^6, 4 \cdot 10^6\}$,
- Super large int (between -2^{31} and $2^{31} - 1$), with $n \in \{10^3, 10^5, 10^6\}$,
- Boltzmann distributed int (in $\llbracket 0, 2^{16} - 1 \rrbracket$), with $n \in \{10^3, 10^5, 10^6, 4 \cdot 10^6\}$,

Remark 19: The datasets above are also used for the benchmarks. The presence of the Boltzmann distribution [2] with parameter $\lambda = 10^{-4}$ was motivated by the dense presence of integers near zero, while making available some rare “big” integers; this observation was suited for testing the overflow implementation.

Additionally, we will also assess the behaviour of our system on some special cases:

- Only one integer,
- Only one integer being zero (to evaluate the behaviour of `__maxBitLength()` and catch potential bugs,

- Only values at the limits of 32-bit encoding are used to see if 32-bit integers are really supported by the system.

To date, all the tests (4 tests passed successfully for all combinations of dataset, special cases and compressors 7x15 + 7x3: 504 individual tests in total) passed successfully.

3.2 – Benchmarks

3.2.1 – Plots

For the benchmarks, we used the same dataset as in the previous section (except for particular cases) and measured the time taken by each method of the compression process on all the compressors (including overflow with different parameters).

The time measurement was made using a custom context manager class, which registers the starting time when opening the context and the ending time when closing the context. The measured time is simply the delta between the two values (elapsed time).

Note that for stability, the time is measured ten times for each dataset and compressor combination. All the results are dumped in separate CSV files (one per classifier, see `benchmark.py` and `OUT/` for more details).

From these files, the first thought we had was to evaluate these elapsed times with respect to the current benchmark file and compressor type. For that, we will first have to aggregate the information into exploitable data and will simply expose the mean of elapsed time for each file and compressor type along with their confidence intervals.

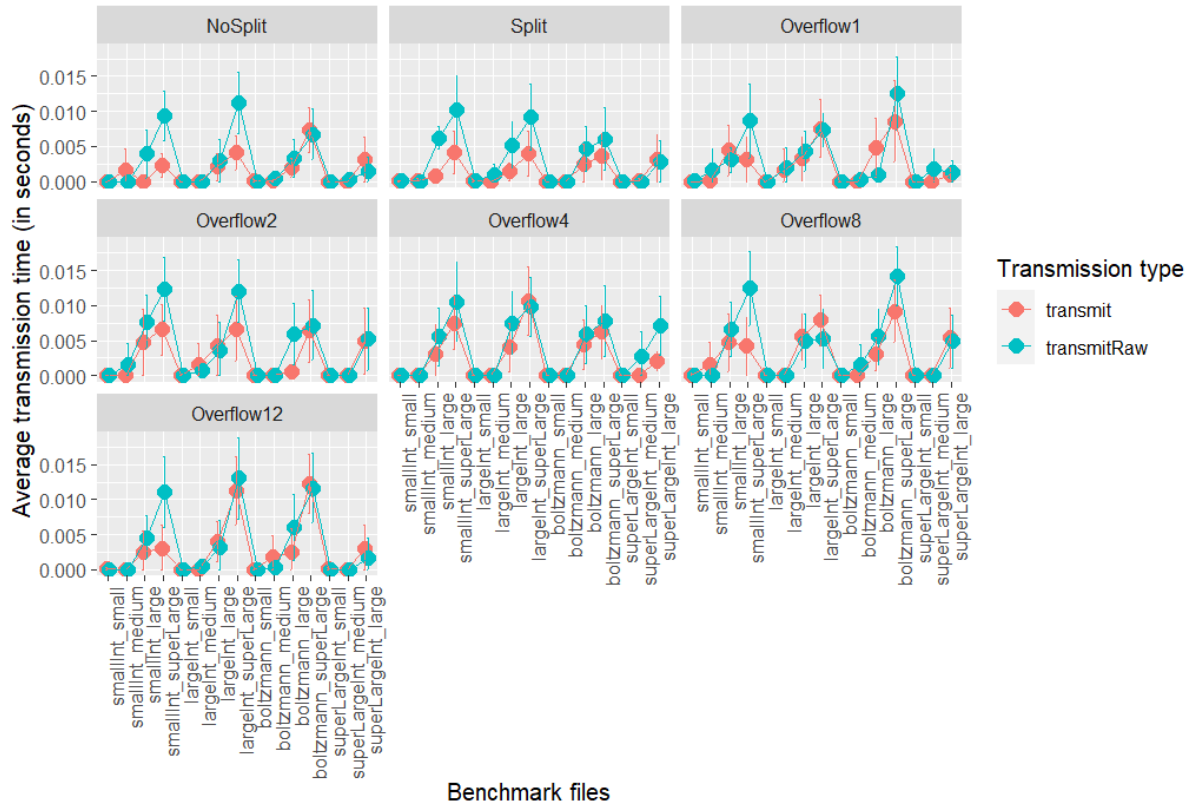
Note: We will compute the confidence intervals using 95% confidence [3], in other words, we will compute:

$$EM_{95\%} = \hat{\mu} \pm 1.96 \frac{\hat{\sigma}}{\sqrt{n}}$$

Where $\hat{\mu}$ is the mean of the sample, $\hat{\sigma}$ the standard deviation of the sample and n the sample size.

Following this idea, we got the following plot:

Average transmission time with respect to the compressor and benchmark output



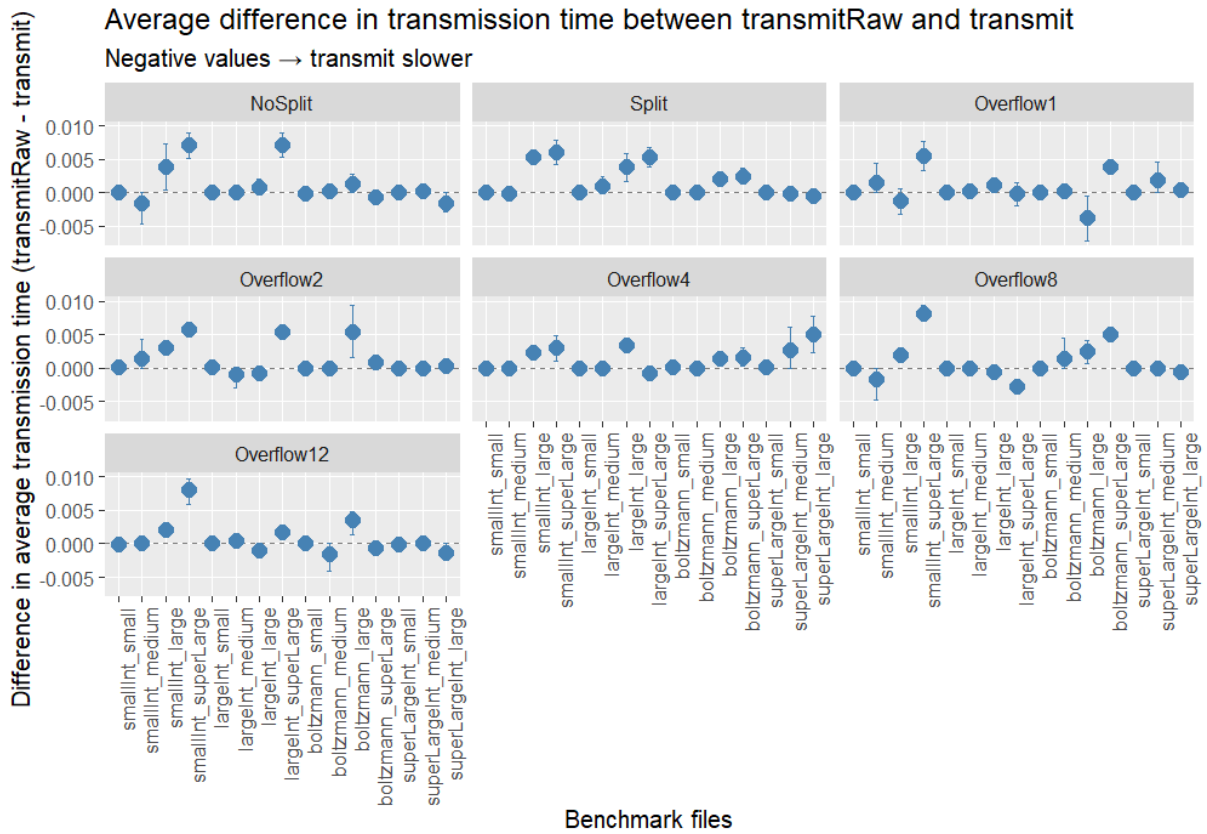
On this plot, we observe that the transmission time, whether we use compressed data or raw data, is quite similar to each other (in terms of absolute scale).

Even though the confidence intervals between raw and compressed transmitted data elapsed time, we can still see a tendency for compressed data to be transmitted faster on average.

This observation is especially true for the no-split compressor on large and super-large datasets of small integers, as well as super-large datasets of large integers, where the confidence intervals do not even touch each other. This observation is also valid for the split compression on a large dataset of small and large integers.

However, with this point of view, it can be difficult to differentiate the values. This constat has motivated the creation of the next plot showing the difference between raw data transmission and compressed data transmission elapsed time:

On this graph, the same data as earlier are shown, but in a clearer manner. Let us now have a look at the differences for each classifier:



No Split compressor: Here, we observe that the transmission time is, most of the time, similar regardless of compression. However, some outliers are observed, both positively and negatively:

Best scenarios:

1. Large and super-large set of small integers,
2. Large and super-large set of large integers,
3. Large set of Boltzmann-distributed integers.

These scenarios, where no split compressor achieves the best performances, correspond to the processing of large-sized sets (between 10^6 and $4 \cdot 10^6$ integers) possessing mainly relatively small integers ($|\text{obs}| \leq 2^8$ or 2^{16}).

Worst scenarios:

1. Medium set of small integers,
2. Super-large set of Boltzmann-distributed integers,
3. Large set of super-large integers.

These scenarios, where no split compressor achieves the worst performances, correspond to large sets (greater than 10^6 integers) of large integers or potentially large integer ($|\text{obs}| \leq 2^{16}$ or 2^{32}). The medium set of small integers is quite unexpected to be found here, and to this date, we cannot really find a good reason apart from randomness to find it in this category.

Apart from the medium-sized set of small integers, the other results clearly reflect what we could have expected with this compressor: small values can be packed on the same 32-bit integer, making the whole array compressed, while larger values need to be shifted to the next 32-bit integer, thus negatively affecting the compression. Additionally, the presence of another boolean array to keep track of the sign may mitigate the performance and explains why we did not observe profound changes for the other values.

Split compressor: Here, we observe that the transmission time is, most of the time, better with compression than without. Here are the scenarios we observed:

Best scenario:

1. Large and super-large set of small integers,
2. Medium, large and super-large set of large integers,
3. Large and super-large set of Boltzmann-distributed integers.

These scenarios, where split compressing achieves the best performances, correspond to the processing of medium to large-sized sets (between 10^5 and $4 \cdot 10^6$ integers) possessing mainly relatively small integers ($|\text{obs}| \leq 2^8$ or 2^{16}).

Worst scenario:

1. Super-large set of super-large integer.

This scenario, where split compressing achieves the worst performances, is characterized by large integers ($|\text{obs}| \leq 2^{32}$) and large dataset $4 \cdot 10^6$.

Here, we also observed something that had to be expected. Indeed, large or small integers can be stacked without the limitation of not splitting the integer; this proved to be more interesting for compression. For very large integers on larger sets, we suspect the inclusion of the bit sign to be at the origin of the deteriorated performances.

Overflow compressor: For this compressor, we will not list each observation but directly make a deduction for the sake of clarity.

On the graph, we observe an overall trend that large datasets result after compression, to better transmission time. This is especially true for small integers that can be stacked and for small overflow threshold; however, for large integers, this observation does not necessarily hold.

Nevertheless, we identified that the threshold efficiency deeply relies on the integer distribution. Indeed, small overflow thresholds result in the best performances, especially for small to medium integers (below 2^{16}), but can also be worth it for larger files possessing large integers. Here, a threshold of 2 or 4 may be considered a sweet spot.

Larger overflow thresholds do not exhibit a particular improvement and can even deteriorate the performances for larger integers by balancing the max bit length on each split compression performed on both normal and overflow area, thus not optimising the split compression for the normal area.

Sum up:

Regardless of the compressor, we identified multiple trends:

1. Compression is usually worth it for larger files,
2. Small integers can be compressed in an easier manner. Thus, rendering a graspable gap compared to before compression transmission,
3. Large integers do not compress well. If we also take into consideration the addition of the sign-bit, then the larger the file, the larger the bit array, thus completely eradicating the benefits for large sets of large integers.

Now, some crucial questions are remaining:

- Is the transmit time significantly smaller than the transmit raw time among all classifiers?
- Is the transmit time significantly smaller than the transmit raw time among all classifiers for large and larger files?
- Is the transmit time significantly smaller than the transmit raw time among all classifiers for medium-sized files?
- Is the transmit time significantly smaller than the transmit raw time among all classifiers and small files?
- **At which dataset size does compression become worth it?**
- Is compression really worth it?

All these questions heavily push us through the question of what is “significant” and heavily imply the usage of hypothesis testing.

In this context, we have the following conditions:

- A. We want to compare two groups (transmit raw data, transmit compressed data),
- B. We do not want to assume normality of the distributions,
- C. We want a test that can manage the presence of zeros (in our data, we have some transmission which are fast enough to be measured as instantaneous),

To meet these requirements, we will use the Fisher-Pitman permutation test on the arithmetic mean [1, 4]. This test supports, by its construction (permutations), the context we posed above.

The hypothesis we will use for the tests are the following:

$$H_0: F_X = F_Y \ (\mu_X = \mu_Y)$$
$$H_1: F_X \text{ is stochastically less than } F_Y \ (\mu_X < \mu_Y)$$

Where F_X is the cumulative distribution function (CDF) of the group X , F_Y the CDF of the group Y , μ_X the mean of the group X and μ_Y the mean of the group Y .

We can perform this test in R using the package `coin` [1] and the `coin::oneway_test` function with `alternative = "less"`.

Q1: Is the transmit time significantly smaller than the transmit raw time among all classifiers?

To answer the first question, we just need to make the groups for transmitting raw, transmitting and selecting the elapsed time:

```
transmitTime <- c(
  as.vector(allClassifier %>% filter(func=="transmit") %>% select(time))$time,
  as.vector(allClassifier %>% filter(func=="transmitRaw") %>% select(time))$time
)
group = as.factor(c(
  rep("transmit", length(transmitTime)%/%2),
  rep("transmitRaw", length(transmitTime)%/%2)
))
coin::oneway_test(transmitTime~group, alternative="less")
```

The result is the following:

Asymptotic Two-Sample Fisher-Pitman Permutation Test

```
data: transmitTime by group (transmit, transmitRaw)
Z = -4.6919, p-value = 1.353e-06
alternative hypothesis: true mu is less than 0
```

To interpret this result, we will simply observe the p-value. Here, $p\text{-value} \ll 5\%$ (standard significance level); therefore, we can say, with a confidence of 95% (and even at 99.9999%), that we can reject the null hypothesis, indicating the same transmitting time with or without compression and conclude that the compressed data are transmitted faster.

This test goes along with our preceding observation and reinforces our confidence in the fact that compression indeed results in faster transmission in general.

Q2: Is the transmitting time significantly smaller than the transmitting raw time among all classifiers for large and larger files?

Here, we repeat the operation of Q1, but we filter to keep only the larger files. As a result, we obtained a p-value of $4.43e-07$ for the largest datasets and 0.003697 for large datasets.

Again, both p-values are under 5%. In consequence, we can reject the null hypothesis that compressing large files does not impact transmission delay at a confidence of 95% and conclude that compressed large and larger files are transmitted faster in general.

Note that, even though we are very confident about the largest files, we are less confident about the large files, but still sufficiently confident to make the preceding conclusion.

This observation is also reflected in the earliest observations on the plots.

Q3: Is the transmitting time significantly smaller than the transmitting raw time among all classifiers for medium-sized files?

Here, the result is a bit different. Indeed, we obtained a p-value of $0.4497 \gg 5\%$. As a consequence, we are not confident enough to reject the null hypothesis and will consider that compressing medium-sized files is not worth it in general.

This result was already observed for some compressors earlier and is therefore not really surprising. This might be explained by the addition of the sign-bit array or the structure used (bitarray, tuple and so on) to render the compressed data, which may not be worth it for a smaller integer array.

Q4: Is the transmitting time significantly smaller than the transmitting raw time among all classifiers and small files?

For the last question, the resulting p-value is $0.2711 \gg 5\%$, which is not sufficient to reject the null hypothesis. We will then consider that small-sized datasets are not necessarily faster to send after compression.

These results can be explained in the same way as Q3, but in an amplified manner, with the addition of structures in the compressed array surrounding.

Q5: At which dataset size does compression become worth it?

Using all these questions, we can build a large interval for which compression seems to be worth it. By considering the tests, compressions seem worth it for large datasets possessing more than integers.

Unfortunately, this answer is not very satisfying since it is quite a large approximation. This is why we also made two linear regressions. The first depicts the transmitting time for raw data, while the second depicts the transmitting time for compressed data.

To do that in R, we transform our dataset a bit and proceed with the following commands, which fit the linear regression and retrieve the coefficient found:

```
lmTransmitRaw <- lm(time~inFile,
  data = allClassifierFile %>% filter(func=="transmitRaw")
)
lmTransmit <- lm(time~inFile,
  data = allClassifierFile %>% filter(func=="transmit")
)
t_coeff <- lmTransmit$coefficients
tr_coeff <- lmTransmitRaw$coefficients
```

In the end, we just need to find the intersection point between the two curves by solving the equation:

$$a + bx = c + dx \Leftrightarrow x = \frac{a - c}{d - b}$$

Where a and c are the intercepts of our linear regressions and b and d the slope of our regressions.

We find:

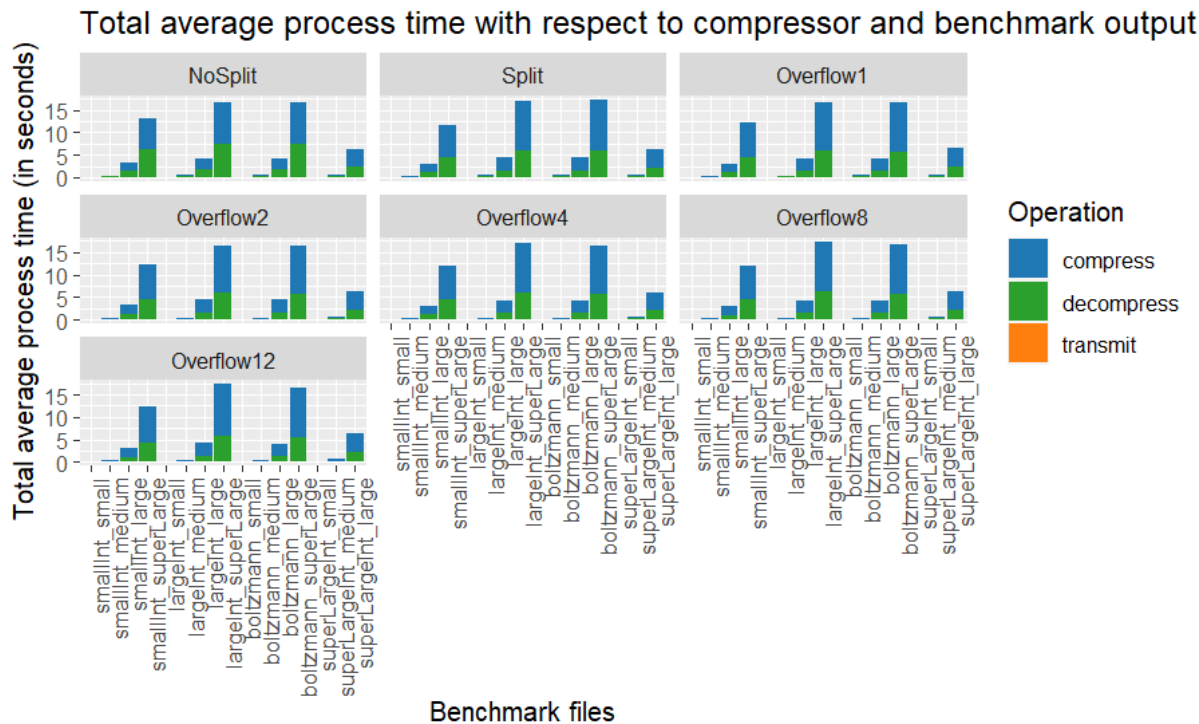
$$\frac{6.739325 \cdot 10^{-4} - 4.778662 \cdot 10^{-4}}{1.589282 \cdot 10^{-9} - 2.425203 \cdot 10^{-9}} = x = -234551$$

We round to the ceiling and obtain a value of -234551 integers. In other words, the compression becomes good on average for files containing at least zero 32-bit encoded integers.

Remark 20: Obviously, the obtained value is not true, as we have seen in our previous observations. These poor results can be explained by an R^2 for both regressions being under 50%, which is a strong signal for a quite terrible regression. As a consequence, the newly found value is not necessarily true even if we specify a particular compressor and find a positive value. As a result, even if the regression can make sense, we will still consider that compression to begin worthy from 1 000 000 32-bit encoded integers.

Q6: Is compression really worth it?

Before concluding, we need to address the elephant in the room. From the beginning of this report, we made all of our analysis on the transmission delay while ignoring the compression delay. But it is a great factor in the total delay. Here is a plot representing the repartitions of the delays:



We will not detail the plot, but we basically see that the compression delay and decompression delay are far more important than the transmission delay, especially for larger arrays.

As a consequence, we advise using these compressors only when the application requires fast transmission for whatever reason but does not impose constraints on the total delay.

Conclusion

In conclusion, we begin this report by describing the architecture of the project, but also the implementation of three distinct types of compression techniques to speed up transmission (reduce transmission delay). These compression techniques split; no split and overflow possess intrinsic particularities which can impact the compression quality.

After implementing and benchmarking, we deduced some properties of our compression techniques and found that compression efficiency improves when increasing the size of our 32-bit encoded integer array and when diminishing the maximum bit-length required to store the biggest absolute value in our array.

Additionally, we found that split compressing yielded the best results among all compressors, but that it could be surpassed by an overflow compression if the threshold were rightfully fixed.

Finally, we determined a quasi-guarantee that compression becomes worth it for all compression techniques in general for a minimum array size of maximum 1 000 000 integers.

Acknowledgement: None

Notice: I declare the usage of Mistral AI and OpenAI chat-bots to exclusively generate the R plot code. Nevertheless, the data cleaning, management, interpretation, and hypothesis testing was entirely made “by-hand.”

Sources

[1] Implementing a Class of Permutation Tests: The coin Package

Authors: Torsten Hothorn, Kurt Hornik, Mark A. van de Wiel, Achim Zeileis

Date of publication: 2008

Journal: Journal of statistical software

Link: <https://rdrr.io/cran/coin/f/inst/doc/Implementation.pdf>

[2] More information on Boltzmann distribution

Source:

Wikipedia

SciPy

Link:

https://en.wikipedia.org/wiki/Boltzmann_distribution

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.boltzmann.html>

[3] More information on Error Margin and Hypothesis Testing

Source:

"ECUE TBTR306 Tests d'hypothèses pour l'analyse bi-variée", François Gautero, BUT2 SD, IUT Nice Côte d'Azur, Université Côte d'Azur

"RES TBTR201 : Statistique inférentielle", Hammel Tarek, BUT1 SD, IUT Nice Côte d'Azur, Université Côte d'Azur

Wikipedia

Link:

None

None

https://en.wikipedia.org/wiki/Margin_of_error

https://en.wikipedia.org/wiki/Statistical_hypothesis_test

[4] The Fisher-Pitman permutation test: an attractive alternative to the F-test

Authors: Kenneth J. Berry, Paul W. Mielke Jr, Howard W. Mielke

Date of publication: 2002

Journal: Psychological Reports 2002

Link: [https://www.researchgate.net/profile/Howard-Mielke/publication/11312045_The_Fisher-](https://www.researchgate.net/profile/Howard-Mielke/publication/11312045_The_Fisher-Pitman_Permutation_Test_An_Attractive_Alternative_to_the_F_Test/links/5f76dc20299bf1b53e074cc1/The-Fisher-Pitman-Permutation-Test-An-Attractive-Alternative-to-the-F-Test.pdf)

[Pitman Permutation Test An Attractive Alternative to the F Test/links/5f76dc2](https://www.researchgate.net/profile/Howard-Mielke/publication/11312045_The_Fisher-Pitman_Permutation_Test_An_Attractive_Alternative_to_the_F_Test/links/5f76dc20299bf1b53e074cc1/The-Fisher-Pitman-Permutation-Test-An-Attractive-Alternative-to-the-F-Test.pdf)

[0299bf1b53e074cc1/The-Fisher-Pitman-Permutation-Test-An-Attractive-Alternative-to-the-F-Test.pdf](https://www.researchgate.net/profile/Howard-Mielke/publication/11312045_The_Fisher-Pitman_Permutation_Test_An_Attractive_Alternative_to_the_F_Test/links/5f76dc20299bf1b53e074cc1/The-Fisher-Pitman-Permutation-Test-An-Attractive-Alternative-to-the-F-Test.pdf)